Description Submission view

Available from: Cuma, 20 Şubat 2026, 10:50 AM
Due date: Cuma, 20 Şubat 2026, 12:15 PM
Requested files: BackupSystem.java (Download)
Type of work: Individual work
Reduction by automatic evaluation: 25 **Free evaluations**: 3**Reduction by automatic evaluation**: 25 **Free evaluations**: 3

Laboratory Task: Distributed Backup System

Objective

In this assignment, you will simulate a **Distributed Backup System**. Your goal is to implement a mechanism that saves a large data file across multiple storage units (Hard Drives). The system requires intelligent data management to handle broken hardware via health checks, safely ignore missing drives, and fragment large files into smaller chunks when necessary.

---

1. Class Structure: `HardDrive`

The `HardDrive` class represents a physical storage unit. It tracks its maximum capacity, current usage, and physical condition.

Important Note: This class must **not** be an inner class. It should be defined as a separate, non-public class in your file.

Attributes:

- `id` (Integer): Unique identifier for the drive.
- `capacity` (Double): The maximum storage space of the drive.
- `usedSpace` (Double): The amount of space currently occupied.
- `health` (Integer): A value (0-100) indicating the drive's reliability.

Constructor:

- Requires `id`, `capacity`, and `health` upon creation.
- The `usedSpace` should be initialized to zero by default.

Methods to Implement:

- `getFreeSpace()`: Returns a Double representing the remaining available capacity on the drive (capacity minus used space).
- `addData(amount)`: Accepts a Double parameter representing the data size to be written. It increases the `usedSpace` of the drive by this amount. Does not return anything.
- `toString()`: Returns a String summarizing the drive's status. For example: "*Drive 1 [Health:90] Used:50.0/100.0*"

---

2. Class Structure: `DataFile`

The `DataFile` class represents the data entity that needs to be backed up into the system.

Important Note: Just like `HardDrive`, this class must **not** be an inner class. It must be a separate, non-public class.

Attributes:

- `size` (Double): The total size of the file to be backed up.
- `isSaved` (Boolean): A status flag indicating if the backup process was fully completed successfully (defaults to false).

Constructor:

- Requires `size` upon creation.
- The `isSaved` flag should be initialized to false.

Core Method: `saveToDrives(drives)` This method (which returns nothing) acts as the backup controller. It takes an array of `HardDrive` objects as a parameter and is responsible for safely distributing the file's size across the provided storage units.

Functional Requirements & Behavior:

1. **Iterate the Array:** You must loop through the `drives` array sequentially (from the first index to the last).
2. **Null Safety:** The storage array might contain empty slots (null references). Ensure your loop safely skips these without throwing errors.
3. **Strict Health Verification:** The system must prioritize data safety. Before writing to any drive, check its health. If a drive's health is strictly **less than 70**, it is considered unstable. The system must skip this drive entirely, print a warning message, and move to the next valid drive.
4. **Data Fragmentation:** A file might be larger than the free space of a single healthy drive. The system must support splitting the file:
   - *Case A:* If the drive has enough free space for the remaining file size, write the exact remaining amount to the drive.
   - *Case B:* If the drive does not have enough free space, fill the drive to its maximum capacity, and carry the remaining unwritten file size over to the next healthy drive in the array.
   - Whenever data is successfully written to a drive, print an informational message indicating the size of the part written and the target drive ID.
5. **Early Exit:** If the entire file is successfully saved (remaining size becomes zero or less) before reaching the end of the array, the loop should terminate early.
6. **Completion Check:** After the iteration finishes (or terminates early), check if the entire file was written. If successful, set `isSaved` to true and print a success message. If the array ran out of space or valid drives before the file could be fully written, print an incomplete message showing the exact amount of data left unsaved.

---

3. Main Class: `BackupSystem`

This is the main, public entry point of your application. You will use this class to create an array of `HardDrive` objects, instantiate a `DataFile`, invoke the backup method, and print the final status of all drives to verify your logic. This must be the **only** `public` class in your file.

---

4. Console Output Requirements

Your `saveToDrives` method must print specific messages to the console during its execution. Please ensure your outputs closely match the following formats to pass the automated tests:

- **When skipping a broken drive:** `"Skip Drive [id]"` (e.g., *Skip Drive 2*)
- **When writing a chunk of data:** `"Part [amount] -> Drive [id]"` (e.g., *Part 50.0 -> Drive 1*)
- **When the file is fully saved:** `"File Saved."`
- **When the storage runs out before saving the whole file:** `"File Incomplete. Left: [remaining amount]"` (e.g., *File Incomplete. Left: 10.0*)

---

5. Example Scenario

The File: Total Size: 100.0 MB

The Storage Array (`drives`):

Array Index	Drive ID	Capacity	Health	System Decision & Action
0	1	50.0 MB	90 (Healthy)	**Write 50.0 MB.** (Drive is now full). Remaining File: 50.0 MB.
1	2	100.0 MB	40 (Unstable)	Health < 70. **Skip.**
2	3	80.0 MB	100 (Healthy)	Has 80 MB space. Need to write 50 MB. **Write 50.0 MB.** Remaining File: 0.0 MB.

Expected Console Output for this Scenario:

```
Part 50.0 -> Drive 1 Skip Drive 2 Part 50.0 -> Drive 3 File Saved.
```

6. Important: Edge Cases & Boundary Conditions To receive full points from the automated tests, your code must explicitly handle these specific scenarios:

- **Boundary Health Values:** A drive with a health score of exactly `70` is considered healthy and must be used. You must only skip drives that are strictly less than 70 (`< 70`).

- **Zero-Byte Files:** If a `DataFile` is created with a size of `0.0`, your method must immediately set `isSaved` to `true` and print "File Saved." without entering the assignment logic or printing any parts.
- **Empty Storage:** If the `drives` array has a length of 0, your code must not crash. It should skip the loop entirely and print the incomplete message ("File Incomplete. Left: [remaining amount]").
- **Exact Fits & Early Exit:** If a file perfectly fills a drive (leaving exactly `0.0` remaining file size), you must immediately stop looking at other drives. Use a `break` statement to exit the loop as soon as the remaining size hits zero or drops below it.

Implementation Notes

- **No Inner Classes:** Do not define `HardDrive` or `DataFile` inside the `BackupSystem` class. They must be separate, non-public classes defined in the same file.
- Do not use advanced collections (like Lists, ArrayLists, etc.); stick strictly to standard arrays as requested.
- Use a local variable inside your method to track the `remaining` file size as you iterate through the array.
- Ensure you check for both null elements and broken drives *before* attempting to calculate space or write data.

```
=== SCENARIO 1: Simple Save & Exact Fills (Tests T18, T19, T30) ===
Part 40.0 -> Drive 1
File Saved.
Part 60.0 -> Drive 1
File Saved.
File Saved.

=== SCENARIO 2: Fragmentation Across Multiple Drives (Tests T21, T22, T29) ===
Part 50.0 -> Drive 2
Part 30.0 -> Drive 3
File Saved.

=== SCENARIO 3: Health Checks - Skips and Exact 70 (Tests T23, T24, T25) ===
Skip Drive 5
Part 50.0 -> Drive 6
File Saved.

=== SCENARIO 4: Null Safety & Empty Arrays (Tests T26, T27, T28) ===
Skip Drive 7
Part 50.0 -> Drive 8
File Saved.
File Incomplete. Left: 10.0

=== SCENARIO 5: Not Enough Space / Incomplete (Tests T20, T34) ===
Part 40.0 -> Drive 9
File Incomplete. Left: 10.0

--- Final Status of All Drives ---
Drive 1 [Health:100] Used:100.0/100.0
Drive 2 [Health:100] Used:50.0/50.0
Drive 3 [Health:100] Used:30.0/50.0
Drive 4 [Health:100] Used:0.0/100.0
Drive 5 [Health:69] Used:0.0/100.0
Drive 6 [Health:70] Used:50.0/100.0
Drive 7 [Health:40] Used:0.0/50.0
Drive 8 [Health:100] Used:50.0/100.0
Drive 9 [Health:100] Used:40.0/40.0
```

Requested files

BackupSystem.java

```java
public class BackupSystem {
    public static void main(String[] args) {

        System.out.println("=== SCENARIO 1: Simple Save & Exact Fills (Tests T18, T19, T30) ===");
        HardDrive[] drives1 = { new HardDrive(1, 100, 100) };

        // File size 40. Fits easily in a 100 capacity drive.
        // Expected: Part 40.0 -> Drive 1 \n File Saved.
        DataFile file1 = new DataFile(40);
        file1.saveToDrives(drives1);

        // File size 60. Fits EXACTLY in the remaining 60 capacity of Drive 1.
        // Expected: Part 60.0 -> Drive 1 \n File Saved.
        DataFile file2 = new DataFile(60);
        file2.saveToDrives(drives1);

        // File size 0.0. Edge case, should save immediately.
        // Expected: File Saved.
        DataFile file3 = new DataFile(0);
        file3.saveToDrives(drives1);


        System.out.println("\n=== SCENARIO 2: Fragmentation Across Multiple Drives (Tests T21, T22, T29) ===");
        HardDrive[] drives2 = {
            new HardDrive(2, 50, 100),
            new HardDrive(3, 50, 100),
            new HardDrive(4, 100, 100) // This drive will be left untouched (Stop Iteration test)
        };

        // File size 80. Fills Drive 2 completely, puts remaining 30 in Drive 3.
        // Expected: Part 50.0 -> Drive 2 \n Part 30.0 -> Drive 3 \n File Saved.
        DataFile file4 = new DataFile(80);
        file4.saveToDrives(drives2);


        System.out.println("\n=== SCENARIO 3: Health Checks - Skips and Exact 70 (Tests T23, T24, T25) ===");
        HardDrive[] drives3 = {
            new HardDrive(5, 100, 69), // Health < 70 (Skip)
            new HardDrive(6, 100, 70)  // Health exactly 70 (Keep)
        };

        // File size 50. Should skip Drive 5 and successfully write to Drive 6.
        // Expected: Skip Drive 5 \n Part 50.0 -> Drive 6 \n File Saved.
        DataFile file5 = new DataFile(50);
        file5.saveToDrives(drives3);


        System.out.println("\n=== SCENARIO 4: Null Safety & Empty Arrays (Tests T26, T27, T28) ===");
        HardDrive[] drives4 = {
            null,
            new HardDrive(7, 50, 40), // Unhealthy (Skip)
            null,
            new HardDrive(8, 100, 100) // Healthy (Keep)
        };

        // File size 50. Skips nulls and unhealthy drive, writes to Drive 8.
        // Expected: Skip Drive 7 \n Part 50.0 -> Drive 8 \n File Saved.
        DataFile file6 = new DataFile(50);
        file6.saveToDrives(drives4);

        // Empty array test
        HardDrive[] emptyDrives = new HardDrive[0];
        // Expected: File Incomplete. Left: 10.0
        DataFile file7 = new DataFile(10);
        file7.saveToDrives(emptyDrives);


        System.out.println("\n=== SCENARIO 5: Not Enough Space / Incomplete (Tests T20, T34) ===");
        HardDrive[] drives5 = { new HardDrive(9, 40, 100) };

        // File size 50. Writes 40 to Drive 9, leaves 10 unsaved.
        // Expected: Part 40.0 -> Drive 9 \n File Incomplete. Left: 10.0
        DataFile file8 = new DataFile(50);
        file8.saveToDrives(drives5);

        System.out.println("\n--- Final Status of All Drives ---");
        printDriveStatus(drives1);
        printDriveStatus(drives2);
        printDriveStatus(drives3);
        printDriveStatus(drives4);
        printDriveStatus(drives5);
    }

    // Helper method to safely print drive statuses
    private static void printDriveStatus(HardDrive[] drives) {
        for(int i = 0; i < drives.length; i++) {
            if(drives[i] != null) System.out.println(drives[i].toString());
        }
    }
}
```

VPL